

从想法 到能跑的产品

30 分钟，跑通”想法 → 能跑的产品”完整路径。

SPEAKER PROFILE

徐爽 · Iris Xu

AI 产品经理 · 5 个 0→1 AI 产品独立作品

CURRENT

01

头部内容/
直播公司

AI 产品经理。主导 2 个 0→1 项目立
项与产品负责。

WORKS

02

5 个独立
AI 作品

Linger · 北极星知识库 · GlowNote ·
Duet · Daily Tarot, 全部 vibe
coding 自驱。

WRITING

03

主编
推荐作者

「人人都是产品经理」 AIPM 方向 4
篇。

EXPERIENCE

04

Vibe Coding
2.5+ 年

2022 末起持续追踪工具链演进。今天
讲的就是我自己跑通的流程。

这个时代变了

01 BEFORE · 过去

想法

→ 找人

→ 排期

→ 几周等

从想法到能看见东西，中间隔着一整支团队和一整个排期表。

- 需要召集人
- 等排期资源
- 反馈循环很慢

02 AFTER · 现在

想法

→ 当天看到

→ 能跑的原型

中间没有人，只有 AI。但前提是——你得先想明白。

- 当天可见
- 反馈即时
- 瓶颈在”讲清楚需求”

vibe coding

"AI 帮我打字"

你想清楚要什么，
AI 实现。

四步走 + 一个心法

从模糊想法到能跑的产品，中间四步。每步配一个主力工具 + 一个关键方法。

STEP 01

调研

这事有人做吗？

STEP 03

原型

长什么样

→ 心法

数据飞轮

收尾揭晓

STEP 02

PRD

想清楚做什么

STEP 04

Kickoff

怎么实现

STEP 01 · RESEARCH

调研

有人做吗？ 他们没做好哪？ 我切哪个口子？

调研不是写报告 是回答三个问题

这事
有人做吗？

01

现状与主要玩家，3-5 家代表性产品的一句话定位。

他们没做好
哪一块？

02

用户公开评论里的高频负面反馈，至少 5 条带出处。

我切哪个
口子？

03

现有产品集体没解决好的问题，3-5 个潜在切入点。

TOOL · KIMI / CHATGPT / PERPLEXITY

派一队 “AI 实习生”分头查

市场01 现状 + 主要玩家 3-5 家代表性产品，每家一句话定位	卖点02 各家核心卖点 差异化优势 + 主打功能或商业模式	痛点03 用户高频抱怨 至少 5 条公开评论 / 应用商店负反馈，带出处	空白04 未被满足的机会 3-5 个潜在切入点，简评切入难度
--	--	---	---

STEP 02 · PRD

苏格拉底 追问

全场最重要的一步。

PROBLEM

“帮我做个 App”

= 让 AI 瞎猜。

- 模糊的需求，产生模糊的产品
- 不是 AI 不行，是没人讲清楚要什么
- 你心里也没讲清楚

让 AI 一次只问一个问题

像剥洋葱，一层一层往下挖。

AI 问01

一次只抛一个具体问题。绝不甩 10 个。



你答02

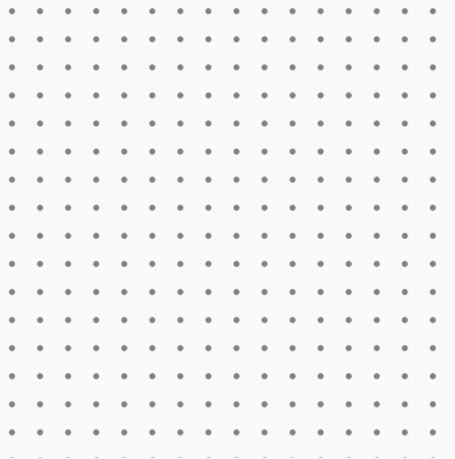
认真答，答不出来也行——说”不确定”它会换角度。



它决定下一个03

顺着你的回答往下挖，挖到你都没意识到的隐含需求。

为什么
“一次一问”
是灵魂



01 一次甩 10 个 → 你会敷衍漏答
人脑应付不了同时多线追问，必然走捷径。

02 顺着回答追问 → 挖出隐含需求
你自己都没意识到的边角约束，被一层层剥出来。

03 模糊需求最适合 → 翻译成规格
“心里有感觉但说不清”恰恰是追问最擅长的场景。

Prompt 2 + 一份 PRD

01 PROMPT 摘录

你是一位资深产品经理。
我有一个模糊想法：[...]

- 规则：
- 1. 一次只问我一个问题。
 - 2. 根据我的回答，决定下一个。
 - 3. 信息足够时，主动停下，输出结构化 PRD。

02 PRD 输出结构

AI 自动生成的合同

追问足够后，AI 主动停下，输出一份结构化 PRD
—— 后面所有环节的合同。

- 项目一句话定位
- 目标用户与典型场景
- 核心痛点
- MVP 核心功能（只列 1 个）
- 成功标准
- 明确不做事

STEP 03 · PROTOTYPE

高保真原型

把 PRD 变成可点击的界面。

做原型最容易翻车的： 顺序反了

STAGE 02

做最重要的一页

核心功能页定下来，组件 / 配色 / 间距全部对齐

STAGE 01

先定视觉设计语言

Pinterest / Dribbble 找 2-3 张参考图，喂给 AI

STAGE 03

以这页为基准铺开

其他页面对齐阶段二，不要每页发挥

STAGE 01

动手前先定 “视觉设计语言”

比起说“高级一点”，
给一张图最有效。

01 找 2–3 张参考截图
Pinterest 或 Dribbble，搜同类产品的优秀界面

02 喂给 AI 让它提炼
提炼共通的视觉风格：色系 / 字体 / 圆角 / 调性

03 拿到 = 设计语言
3–5 关键词 + 色板 + 字体方案 → 之后所有页面对齐

FROM ONE PAGE TO A SET

先把最难的一页做到位 再铺开

02 STAGE 02

做最重要 那一页

在这一页上把视觉、组件、间距、配色全部定下来，
作为后面所有页面的基准。

- 核心功能页(MVP 对应)
- 严格按阶段一设计语言
- 真实内容，不要 Lorem ipsum
- 可点击的核心交互

03 STAGE 03

以这页为基准 铺开

其他页面只复用、不重新发挥。整套原型才会统一、
可信。

- 组件 / 间距 / 配色全部对齐
- 不要每页发挥不同风格
- 按 PRD 的功能与流程铺开

原型改了 → PRD 也要改

原型几轮调整后，旧 PRD 已过时。必须用 Prompt 4 重塑成最新版，不要新旧并存。

V1

旧 PRD

原型阶段之前的版本

V2

最新 PRD

完全对齐当前原型

迭代

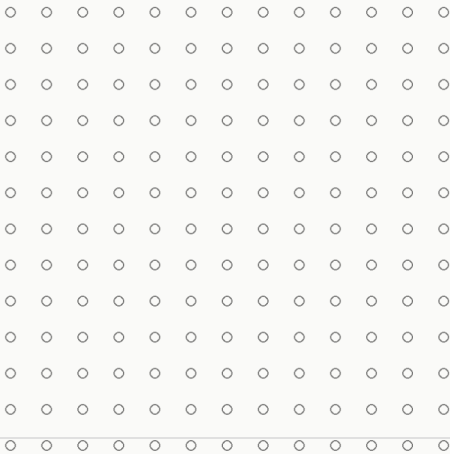
原型调整

加页面 / 改流程 / 砍功能

STEP 04 · KICKOFF

Kickoff

不是手写 CLAUDE.md,
是让 AI 给自己写入职手册。



WHAT IT IS

CLAUDE.md = 给 AI 的入职手册

每次 Claude Code 工作前，先读这份文件，按里面的规矩做事。

A UPPER · 项目事实

它是什么

让 Code 在 Kickoff 阶段自己填，你不用手写。

- 项目一句话定位
- 技术栈与依赖
- 目录结构
- 启动 / 测试 / lint 命令
- 环境变量与密钥
- 不要碰的文件

B LOWER · 工作规范

怎么干活

普适规则，所有项目通用，原样保留。

- 先讲逻辑再动手
- 改动分级 A / B / C
- 即时 git，保证能回退
- 保持代码健康
- API 与密钥安全
- ...共 11 条

四步开工 · CLAUDE.md 自动生成

02

搭脚手架

初始化 + 装依赖，不写功能

04

首个 git commit

.gitignore + .env.example + 入
库

01

提议技术栈

主语言 / 框架 / 目录，等你确
认

03

/init 填项目事实

CLAUDE.md 上半自动补全

一份能落地的 CLAUDE.md

判断不准时，
向高一级靠。

A

出错回不去 / 碰核心资产

数据库 · 密钥 · 支付 · force push · 跨模块重构 → 必须先写方案，等明确”可以”

B

用户能感知但可回滚

UI / 文案 / 新功能 / 模块内业务逻辑 → 一句话说意图，做完测+lint，单独 commit

C

纯局部 / 可逆 / 不影响运行

注释 / 改名 / 加日志 / 加测试 / 修 typo → 直接做，一起 commit

DON'T PANIC

看到不懂的词，不要慌

不要无视。不要畏难。去查。

lint

01

代码格式检查工具，自动挑出写得不规范的地方

typecheck

02

类型检查，在运行前发现”用错类型”的 bug

Conventional Commits

03

commit message 的标准格式：feat / fix / refactor 等

force push

04

强制推送，会覆盖远端历史 — 危险操作，A 级改动

DRY

05

Don't Repeat Yourself · 不要重复造轮子

.gitignore

06

告诉 git 忽略哪些文件（密钥、临时文件都靠它）

今天不懂的词，[三个月后是你随口就能用的工具。](#)

SHORTCUT

边写边沉淀

`#` 键

只沉淀长期规则。
不沉淀临时琐事。

01 输入框打 `#`

跑测试前必须先问用户确认

02 自动并入 CLAUDE.md

Claude Code 把这条变成永久规则，加在文件适当位置。

03 下次会话已经”懂”

不用再纠正同一件事 — 这就是飞轮的字面机制。

四步串起来

- STEP 01

调研

想清楚
做不做

- STEP 02

追问

想清楚
做什么

- STEP 03

原型

确认
长什么样

- STEP 04

Kickoff

解决
怎么实现

CLOSING · 01

别追求快。

第一次都会很慢，会很挫败。
这很正常 — 别人也一样。

— 但你会越来越快。

DATA FLYWHEEL

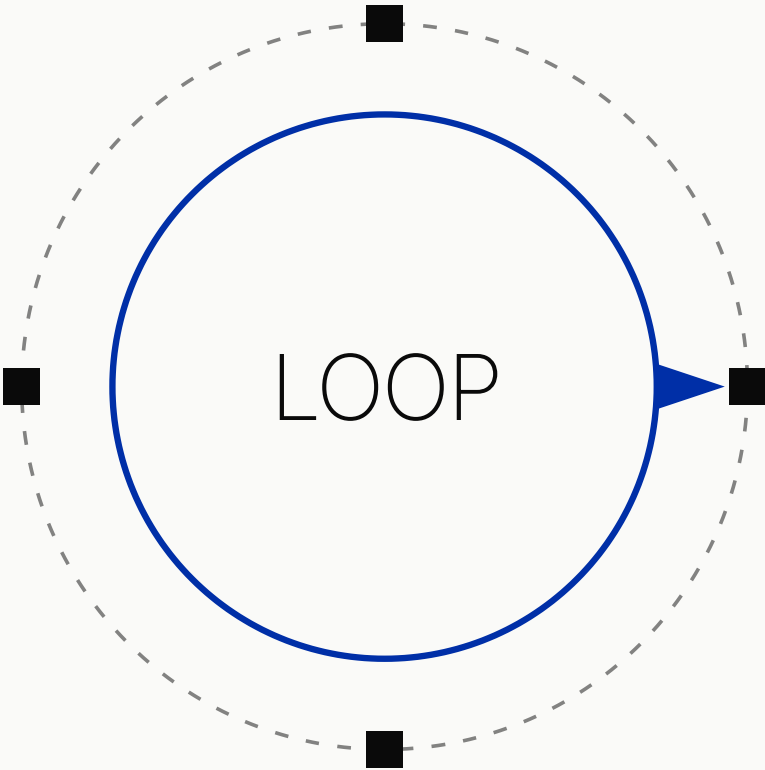
这是你自己的数据飞轮

01 踩坑 · 查不懂的词

02 按 `#` 沉淀新规则

03 CLAUDE.md 长出来

04 下一个项目更快



会用 AI 写代码
不稀奇。

愿意慢下来，
把飞轮转起来的人
才会真正甩开
别人。

01 先想清楚，再动手
瓶颈不是代码，是讲不清楚要什么。

02 CLAUDE.md = 入职手册
让 AI 给自己写规则，你审核，大家共享。

03 转你自己的飞轮
第一次的慢，是飞轮启动必付的成本。